BEGINNER-FRIENDLY GUIDE

Employee Management Project

A simple, file-by-file guide to what the application does and how its parts work together.

> **In one sentence:** This is a Spring Boot REST API that lets a client such as Postman create, list, view, update, and delete employee records stored in a MySQL database.

How a request travels through the app

Postman / client sends a URL, HTTP method, and possibly JSON
↓
EmployeeController receives the request
↓
EmployeeRequest checks that input is valid
↓
EmployeeServiceImpl applies the employee rules
↓
EmployeeRepository reads or writes data
↓
MySQL stores the employee row

The result travels back in reverse: the service turns a database **Employee** into an **EmployeeResponse**, and the controller returns it as JSON. If input or business rules fail, **GlobalExceptionHandler** turns the error into a readable HTTP response.

Project folders at a glance

Location	What is inside
`src/main/java`	Application code: API controller, DTOs, service, repository, entity, and exceptions.
`src/main/resources`	Application settings, including database connection and server port.
`src/test/java`	Automated test source files.
`.mvn`, `mvnw`, `mvnw.cmd`	Maven Wrapper files for running Maven without installing it separately.
`target`	Generated build output. Maven creates it; do not edit its contents by hand.

> **Package note:** Java package names are declared inside each source file, for example `com.example.employeemanagement.controller`. The package declaration determines the Java package even if directory names look shorter.

This guide covers all application files in the Maven project folder. IDE settings and generated build output are not application source.

USING THE API

Endpoints and Postman examples

Method	URL	What it does	Usual success
POST	`/api/employees`	Create an employee	201 Created
GET	`/api/employees`	Get all employees	200 OK
GET	`/api/employees/{id}`	Get one employee by ID	200 OK
PUT	`/api/employees/{id}`	Replace an employee's details	200 OK
DELETE	`/api/employees/{id}`	Delete an employee	204 No Content

Create an employee

In Postman select **POST**, enter `http://localhost:8080/api/employees`, choose **Body** → **raw** → **JSON**, and send:

```
{
  "firstName": "Asha",
  "lastName": "Rao",
  "email": "asha@example.com",
  "department": "Engineering",
  "salary": 65000.00
}
```

All five fields are required. Email must look like an email address, and salary must be greater than zero.

What if first name is missing?

The controller uses `@Valid`, so Spring checks the rules before calling the service. Missing first name returns HTTP **400 Bad Request** and a body similar to:

```
{
  "status": 400,
  "error": "Validation Failed",
  "message": "First name is required",
  "fieldErrors": {
    "firstName": "First name is required"
  }
}
```

Why did a 405 happen?

HTTP 405 means the URL exists but the selected HTTP method is not allowed there. Employee creation uses **POST** at `/api/employees`. PUT and DELETE need an ID, such as `/api/employees/4`. A wrong method is rejected before first-name validation runs.

JAVA FILES

Application, controller, and DTO files

`EmployeeManagementApplication.java`

This is the starting point of the Spring Boot app. Its `main` method starts Spring. `@SpringBootApplication` tells Spring to configure the app and scan for its controllers, services, repositories, and other components.

`EmployeeController.java`

This file defines the web API. `@RestController` means its methods return HTTP responses, usually JSON. `@RequestMapping("/api/employees")` sets the common URL prefix. Each endpoint method delegates real work to `EmployeeService`.

- `createEmployee`: accepts JSON, validates it, calls the service, and returns status 201.
- `getAllEmployees`: returns the employee list.
- `getEmployeeById`: reads the ID from the URL and returns one employee.
- `updateEmployee`: validates replacement data and updates the selected ID.
- `deleteEmployee`: deletes the selected ID and returns an empty 204 response.

`@RequestBody` converts JSON into a Java object. `@PathVariable` reads a value such as `4` from the URL. `@Valid` asks Spring to check validation rules.

`EmployeeRequest.java`

This is the input shape for POST and PUT. It keeps incoming data separate from the database object.

- `firstName`, `lastName`, and `department`: cannot be null, empty, or just spaces; maximum 100 characters.
- `email`: required, must look like an email, maximum 150 characters.
- `salary`: required and greater than zero.

Getters and setters let Spring fill in and read the fields from JSON.

`EmployeeResponse.java`

This is the output shape sent back to the client. It contains the generated ID and employee details, with getters and setters so Spring can convert it to JSON. Returning a response DTO avoids exposing the database entity directly.

JAVA FILES

Service, repository, and database entity

`EmployeeService.java`

This interface lists the operations the app supports: create, read all, read one, update, and delete. It names the work without specifying how it is done. The controller depends on this interface.

`EmployeeServiceImpl.java`

This class implements the service operations. `@Service` lets Spring create and inject it. `@Transactional` makes database work run in transactions.

Part	Plain-language job
`toResponse`	Copies entity values into the API response DTO.
`createEmployee`	Trims and lowercases email, checks for duplicates, copies request values into an entity, saves it, and returns the response.
`getAllEmployees`	Loads all rows and converts each one to a response DTO. It is read-only.
`getEmployeeById`	Loads one row. If none exists, throws `EmployeeNotFoundException`.
`updateEmployee`	Finds the row, checks email uniqueness, updates fields, saves, and returns the response.
`deleteEmployee`	Finds and deletes the row. A missing ID raises the not-found exception.

`EmployeeRepository.java`

This interface connects the service to the database. It extends Spring Data JPA's `JpaRepository`, which supplies common operations like `findAll`, `findById`, `save`, and `delete`. Spring creates the implementation at runtime.

`existsByEmail` and `existsByEmailAndIdNot` are derived queries. Spring reads their names and builds database checks for duplicate email addresses.

`Employee.java`

This is the database entity. `@Entity` marks it as a table-backed class, and `@Table(name="employees")` sets the table name. Its fields are ID, first name, last name, email, department, and salary.

- `@Id` marks the primary key; `@GeneratedValue` lets MySQL assign it.
- `@Column(nullable=false)` requires a database value; length and decimal settings define storage limits.
- Email is unique, so the database also prevents duplicates.
- The no-argument constructor and getters/setters help JPA load and save this object.

Exception files

`GlobalExceptionHandler.java`

`@RestControllerAdvice` makes this class a central place to turn exceptions into JSON responses. API methods can throw exceptions, and this class converts them into readable HTTP errors.

Handler	When it runs	Status
`handleNotFound`	An employee ID does not exist.	404 Not Found
`handleDuplicate`	An email is already in use.	409 Conflict
`handleValidation`	A request field fails a DTO rule.	400 Bad Request
`handleMethodNotAllowed`	The URL was called using a method it does not accept.	405 Method Not Allowed

The validation handler puts field names and messages in `fieldErrors` and joins those messages into `message`. The method-not-allowed handler reports supported methods when Spring knows them and preserves the HTTP `Allow` header. The private `error` helper creates the shared status, error, and message fields for not-found and duplicate-email cases.

`EmployeeNotFoundException.java`

A custom exception used when an employee ID is missing. It carries a message such as “Employee not found with id: 12.” The global handler returns it as a 404 response.

`DuplicateEmailException.java`

A custom exception used when an email is already taken. The global handler returns its message as a 409 response.

> **Remember:** A missing first name is checked before the service runs. A duplicate email is checked inside the service after input validation passes.

CONFIGURATION

Settings and Maven files

`application.properties`

This file holds settings Spring reads when the app starts.

- `spring.application.name`: names the app in logs and tools.
- `spring.datasource.url`: connects to the local MySQL database named `employee_db`.
- `DB_USERNAME` and `DB_PASSWORD`: environment variables can override the local defaults.
- `spring.jpa.hibernate.ddl-auto=update`: Hibernate creates or updates the table structure at startup.
- `spring.jpa.show-sql` and `format_sql`: print readable database statements for development.
- `server.port=8080`: the app listens on port 8080.

> MySQL must be running and `employee_db` must be available before the app can connect. Keep real passwords out of shared source files.

`pom.xml`

This is Maven's project recipe. It identifies the app, inherits Spring Boot dependency versions from its parent, sets Java 17, and lists required libraries.

- Spring Data JPA: database mapping and repository support.
- Spring Validation: checks rules such as `@NotBlank`.
- Spring Web MVC: REST endpoints and HTTP/JSON support.
- MySQL Connector/J: connects to MySQL at runtime.
- Spring Boot test starters: test libraries.
- Spring Boot Maven Plugin: packages and runs the Spring Boot app.

`.mvn/wrapper/maven-wrapper.properties`

Chooses the Maven version used by the wrapper. This project uses Maven Wrapper 3.3.4 and Apache Maven 3.9.11. The 3.9.11 choice avoids the Maven 3.10.0 IntelliJ importer issue seen earlier.

`mvnw and mvnw.cmd`

These are Maven Wrapper launchers. `mvnw` is for macOS/Linux shells; `mvnw.cmd` is for Windows. They download the configured Maven version if needed, then run Maven. On Windows, from this folder, run `.\mvnw.cmd spring-boot:run`.

PROJECT SUPPORT FILES

Tests, help, and version-control files

`EmployeeManagementApplicationTests.java`

This is a Spring Boot context-load test. Its `contextLoads` test starts the application context and passes if Spring can create its components. Because this app uses MySQL, the test may need a working database configuration.

`EmployeeServiceImplTest.java`

This file currently contains an empty test class. It has no test methods yet, so it does not currently check service behavior.

`.gitignore`

This tells Git which files to leave out when saving project changes. It excludes generated build folders such as `target/`, IDE settings, and temporary files, keeping machine-specific or generated files out of source control.

`.gitattributes`

This asks Git to keep line endings consistent: the Unix wrapper uses LF, and Windows `.cmd` files use CRLF. Consistent line endings reduce noisy changes across operating systems.

`HELP.md`

Spring Initializr generated this help file. It links to Maven, Spring Boot, Web MVC, JPA, MySQL, and validation guides. It is documentation, not code needed at runtime.

Other folders and files

- `target/`: generated classes and packaged output from Maven. It can be recreated by building the app, so do not edit it manually.
- `.idea/`: IntelliJ's local project/workspace settings. These are for the editor and do not implement the API.
- No README is present in the app folder; this PDF gives a beginner-friendly project overview.

Quick glossary

Word	Easy meaning
API	A set of URLs and rules another program can use.
DTO	A small object used to receive or return data.
Entity	A Java object that represents a database row.
Repository	The data-access layer that talks to the database.
Validation	Checks that incoming values follow rules before saving.
HTTP status	A number describing the result: 200 success, 400 invalid input, 404 missing record, 409 duplicate, 405 wrong method.

> **Typical create flow:** Start MySQL → run `.\mvnw.cmd spring-boot:run` → send POST in Postman → Spring validates JSON → service checks rules → repository saves it → response JSON comes back.

End of guide.